

Rockchip Linux Memory Trouble Shooting文档

文件标识: RK-KF-YF-C36

发布版本: V1.1.0

日期: 2025-01-22

文件密级: ☐绝密 ☐秘密 ☒内部资料 ☐公开

免责声明

本文档按“现状”提供, 瑞芯微电子股份有限公司(“本公司”, 下同)不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因, 本文档将可能在未经任何通知的情况下, 不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标, 归本公司所有。

本文档可能提及的其他所有注册商标或商标, 由其各自所有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴, 非经本公司书面许可, 任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部, 并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: www.rock-chips.com

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: fae@rock-chips.com

前言

概述

本文档是处理Memory相关问题思路的汇总

产品版本

芯片名称	内核版本
ROCKCHIP 芯片	6.1

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.1.0	薛小明	2025-1-22	初始发布

目录

Rockchip Linux Memory Trouble Shooting文档

1. log出现swiotlb buffer is full的错误信息
2. 如何获取32bit以内的物理地址
3. kmalloc能不能指定32bit物理地址
4. 如何dump内核页表
5. dma_alloc_attrs如何执行
6. io命令访问不了物理内存
7. 内核如何预留CMA内存
8. 如何为设备预留内存
9. 如何使用dma-heap
10. 如何映射内核态内存
11. 如何映射物理内存到用户态地址
12. 内存泄漏分析
13. 冲内存分析
14. 如何获取symbol的物理地址
15. 修改系统内存
16. 内存回收策略调整
17. 内核各级页表物理地址
18. slab相关问题分析
19. 内核线性映射的cache属性
20. 如何将用户态的地址转换成物理地址
21. io读回来的数据存在一致性问题
22. ARM32环境预留奇数M的CMA引起内核奔溃
23. 用户态访问PCIe BAR空间异常
24. dts当中预留内存no-map什么意思
25. 利用/dev/mem访问物理内存
26. 内存一致性问题
27. write-combined方式映射的内存存在一致性问题
28. dma_alloc_coherent失败
29. DRM如何申请带cache的buffer
30. dma_buf fd的使用方法
31. 生成dma_buf fd的方法
32. 如何配置dma-coherent设备
33. PCIe设备如何继承PCIe控制器的dma-coherent属性
34. 如何分析CMA内存是否被系统借用
35. 设置CMA不参与系统内存分配
36. 透明大页TRANSPARENT_HUGEPAGE
37. 无法从缓存中回收内存
38. IOVA申请失败

1. log出现swiotlb buffer is full的错误信息

默认swiotlb size为IO_TLB_DEFAULT_SIZE(64MB)，通过内核命令行传参数修改swiotlb slot个数来增大swiotlb size，比如swiotlb=0x10000，对应的swiotlb size就是 $(0x10000 \ll IO_TLB_SHIFT) \gg 20 = 128MB$ 。swiotlb单次允许申请连续的空间是 $IO_TLB_SEG_SIZE * IO_TLB_SIZE = 256KB$ ，当设备一次申请的buffer超过256K也会报错，当遇到该问题时可以先调整IO_TLB_SEG_SIZE来确认问题，最终还是要通过调整设备驱动来解决。设备的寻址空间如果和内存地址总线位宽匹配时，就需要借助swiotlb的内存。比如设备只支持32bit即4GB的内存地址，但是设备驱动获取到的内存地址为40bit，这时候就需要借助swiotlb来进行地址转换，一般是通过dma_map_single/dma_unmap_single等内核接口函数实现，这个过程会自动拷贝数据以及维护cache一致性，因此会影响性能。当设备内存超过4G达到8G或者16G时，设备驱动更容易出现swiotlb buffer is full的错误，原因是很多时候系统优先从高于32位的地址申请内存，因此会导致swiotlb更容易被消耗。

2. 如何获取32bit以内的物理地址

1. 使用dts的方式预留
2. 使用GFP_DMA32参数,例如:

```
alloc_page(GFP_DMA32 | GFP_KERNEL)
__get_free_pages(GFP_DMA32, 0)
```

需要注意的是dma_alloc_coherent之类的函数不支持直接设置GFP_DMA32的参数，dma_alloc_coherent在执行过程中会自动调整flag。

3. kmalloc能不能指定32bit物理地址

新的内核kmalloc不能指定GFP_DMA32来获取32bit以内的物理地址。经常看到设备驱动使用kmalloc申请内存，然后将物理地址配置给硬件，因为kmalloc的地址和硬件不匹配，导致硬件工作异常，针对这种情况可以自己创建slab来解决，例子如下：

```
struct kmem_cache *mem_cache = kmem_cache_create("test", size, 0,
SLAB_HWCACHE_ALIGN, 0);
void *p = kmem_cache_zalloc(mem_cache, GFP_DMA32 | GFP_KERNEL);
```

p就是从32bit物理内存获取的。

4. 如何dump内核页表

内核打开CONFIG_PTDUMP_CORE

cat /sys/kernel/debug/kernel_page_tables便会输出内核页表信息

5. dma_alloc_attrs如何执行

dma_alloc_attrs执行比较复杂，有两条执行路径，和设备的dts是否引用iommu节点有关

1. 引用iommu

```
dma_alloc_attrs -> iommu_dma_ops.alloc -> iommu_dma_alloc
```

2. 没有引用iommu

```
dma_alloc_attrs -> dma_direct_alloc
```

需要说明的是iommu_dma_alloc申请的物理内存不一定是连续的，但是返回给设备连续的IOVA,如果有遇到内存分配相关的问题，可以根据实际情况跟踪函数

6. io命令访问不了物理内存

CONFIG_DEVMEM打开

CONFIG_STRICT_DEVMEM关闭

7. 内核如何预留CMA内存

以下两种方式

1. 配置CONFIG_CMA_SIZE_MBYTES.
2. dts添加如下:

```
reserved-memory {
    #address-cells = <2>;
    #size-cells = <2>;
    ranges;

    mem_reserved: mem-rsv@40000000 {
        reg = <0x00000000 0x40000000 0x00000000 0x00100000>;
        compatible = "shared-dma-pool";
        reusable;
        linux,cma-default;
    };
};
```

当dts配置了linux,cma-default;CONFIG_CMA_SIZE_MBYTES将失效
内核代码所在位置: kernel/dma/contiguous.c

8. 如何为设备预留内存

dts做如下修改:

```
reserved-memory {
    #address-cells = <2>;
    #size-cells = <2>;
    ranges;

    mem_reserved: mem-rsv@40000000 {
        reg = <0x00000000 0x40000000 0x00000000 0x00100000>;
        compatible = "shared-dma-pool";
        no-map;
    };
};

node: np@0 {
    memory-region = <&mem_reserved>;
};
```

内核代码所在位置: kernel/dma/coherent.c

设备驱动的probe函数添加如下补丁:

```
of_reserved_mem_device_init(dev);
```

0x40000000 - 0x40100000这1MB空间的内存空间由node设备独享, 设备驱动使用

dma_alloc_attrs/dma_alloc_free从这1M空间申请/释放内存。

需要注意的是不能针对这段空间做cache同步操作, 原因是no-map的空间不会建立内核线性映射, 而cache同步操作依赖内核线性映射。

9. 如何使用dma-heap

dma-heap依赖dma-buf, dma_buf_export + dma_buf_fd生成基于dma-buf的fd, 用户态基于该fd访问内存。以cma_heap为例:

用户态首先通过/dev/dma_heap/cma获取到dma_buf的fd, 然后基于该fd访问内存,大致路径如下:

dma_buf_fops -> cma_heap_buf_ops。具体参考内核测试代码tools/testing/selftests/dmabuf-heaps/dmabuf-heap.c

10. 如何映射内核态内存

```
void *vmap(struct page **pages, unsigned int count, unsigned long flags,
pgprot_t prot)
```

其中pages存储的是需要映射的page页

11. 如何映射物理内存到用户态地址

```
int remap_pfn_range(struct vm_area_struct *vma, unsigned long addr, unsigned
long pfn, unsigned long size, pgprot_t prot)
```

remap_pfn_range一般是在mmap回调中使用，pfn物理页帧，假设物理地址是phys，pfn = phys >> PAGE_SHIFT;

可以参考内核中很多remap_pfn_range的实现方法

12. 内存泄漏分析

内核打开如下配置：

CONFIG_DEBUG_KMEMLEAK_MEM_POOL_SIZE=50000

CONFIG_DEBUG_KMEMLEAK=y

mount -t debugfs nodev /sys/kernel/debug/

1. 开始:

echo clear > /sys/kernel/debug/kmemleak

echo scan > /sys/kernel/debug/kmemleak

2. 运行被观察对象

3. 结束:

cat /sys/kernel/debug/kmemleak

内核提供了测试代码,如下:

打开CONFIG_SAMPLES

打开CONFIG_DEBUG_KMEMLEAK_TEST

串口会输出堆栈信息,如下:

```
unreferenced object 0xffff8001085a3000 (size 1024):
  comm "swapper/0", pid 1, jiffies 4294892470 (age 66.712s)
  hex dump (first 32 bytes):
    00 00 00 00 01 00 00 00 08 30 5a 08 01 80 ff ff  ....0Z....
    08 30 5a 08 01 80 ff ff 00 00 00 00 00 00 00 00  .0Z.....
  backtrace:
    [<00000000b428bb03>] slab_post_alloc_hook+0x90/0x308
    [<0000000044e70370>] __kmem_cache_alloc_node+0x15c/0x224
    [<000000004145f605>] kmalloc_trace+0x4c/0x114
    [<00000000ec1ffc0b>] kmemleak_test_init+0x80/0x2b8
    [<000000005abaeb69>] do_one_initcall+0xc4/0x2cc
    [<00000000637fb484>] do_initcall_level+0x9c/0x140
    [<00000000c6cdcb23>] do_initcalls+0x54/0x94
    [<000000006662279d>] do_basic_setup+0x1c/0x28
    [<00000000b072decb>] kernel_init_freeable+0xe0/0x158
    [<00000000fe8f4d8d>] kernel_init+0x20/0x1a4
    [<00000000d1a4f820>] ret_from_fork+0x10/0x20
```

13. 冲内存分析

内核打开如下配置:

CONFIG_KASAN=y

CONFIG_SLUB_DEBUG=y

用以下代码构造越界访问:

```
char *buf = kmalloc(64, GFP_KERNEL);
buf[64] = 'A';
```

串口会输出堆栈信息,如下:

```
[ 7.280718][ T1]
=====
[ 7.280742][ T1] BUG: KASAN: slab-out-of-bounds in kasan_test+0x3c/0x60
[ 7.280773][ T1] Write of size 1 at addr ffff80010d02f540 by task
swapper/0/1
[ 7.280794][ T1]
[ 7.280809][ T1] CPU: 4 PID: 1 Comm: swapper/0 Not tainted 6.1.115 #389
[ 7.280832][ T1] Hardware name: Rockchip RK3588 EVB1 LP4 V10 Board (DT)
[ 7.280849][ T1] Call trace:
[ 7.280863][ T1] dump_backtrace+0xf4/0x114
[ 7.280890][ T1] show_stack+0x18/0x24
[ 7.280913][ T1] dump_stack_lvl+0x6c/0x90
[ 7.280935][ T1] print_report+0x15c/0x4a0
[ 7.280957][ T1] kasan_report+0xac/0x100
[ 7.280977][ T1] __asan_store1+0x6c/0x70
[ 7.280997][ T1] kasan_test+0x3c/0x60
[ 7.281017][ T1] do_one_initcall+0xe4/0x3d4
[ 7.281038][ T1] do_initcall_level+0xd4/0x1e8
[ 7.281061][ T1] do_initcalls+0x54/0x98
[ 7.281082][ T1] do_basic_setup+0x64/0x78
[ 7.281103][ T1] kernel_init_freeable+0x144/0x1c0
[ 7.281125][ T1] kernel_init+0x20/0x1b4
[ 7.281146][ T1] ret_from_fork+0x10/0x20
[ 7.281167][ T1]
[ 7.281180][ T1] Allocated by task 1:
[ 7.281196][ T1] kasan_set_track+0x4c/0x80
[ 7.281216][ T1] kasan_save_alloc_info+0x24/0x30
[ 7.281238][ T1] __kasan_kmalloc+0xa0/0xb8
[ 7.281258][ T1] kmalloc_trace+0x68/0x120
[ 7.281280][ T1] kasan_test+0x30/0x60
[ 7.281300][ T1] do_one_initcall+0xe4/0x3d4
[ 7.281320][ T1] do_initcall_level+0xd4/0x1e8
[ 7.281341][ T1] do_initcalls+0x54/0x98
[ 7.281362][ T1] do_basic_setup+0x64/0x78
[ 7.281383][ T1] kernel_init_freeable+0x144/0x1c0
[ 7.281405][ T1] kernel_init+0x20/0x1b4
[ 7.281425][ T1] ret_from_fork+0x10/0x20
[ 7.281445][ T1]
```



```

[ 7.281458][ T1] The buggy address belongs to the object at
ffff80010d02f500
[ 7.281458][ T1] which belongs to the cache kmalloc-64 of size 64
[ 7.281479][ T1] The buggy address is located 0 bytes to the right of
[ 7.281479][ T1] 64-byte region [ffff80010d02f500, ffff80010d02f540)
[ 7.281502][ T1]
[ 7.281515][ T1] The buggy address belongs to the physical page:
[ 7.281530][ T1] page:000000001d71e131 refcount:1 mapcount:0
mapping:0000000000000000 index:0x0 pfn:0x4340b
[ 7.281553][ T1] flags: 0x4000000000000200 (slab|zone=1)
[ 7.281579][ T1] raw: 4000000000000200 0000000000000000 dead000000000122
ffff800100004280
[ 7.281602][ T1] raw: 0000000000000000 0000000080800080 00000001ffffffff
0000000000000000
[ 7.281619][ T1] page dumped because: kasan: bad access detected
[ 7.281635][ T1]
[ 7.281647][ T1] Memory state around the buggy address:
[ 7.281665][ T1] ffff80010d02f400: fa fb fb fb fb fb fb fb fc fc fc fc fc
fc fc fc
[ 7.281683][ T1] ffff80010d02f480: fa fb fb fb fb fb fb fb fc fc fc fc fc
fc fc fc
[ 7.281702][ T1] >ffff80010d02f500: 00 00 00 00 00 00 00 00 fc fc fc fc fc
fc fc fc
[ 7.281718][ T1]
[ 7.281736][ T1] ffff80010d02f580: fc fc fc fc fc fc fc fc fc fc fc fc fc
fc fc fc
[ 7.281754][ T1] ffff80010d02f600: fc fc fc fc fc fc fc fc fc fc fc fc fc
fc fc fc
[ 7.281771][ T1]
=====

```

14. 如何获取symbol的物理地址

symbol指函数,全局变量等。

假设函数名是function, function对应的物理地址是__pa_symbol(function);

假设全局变量名是val, val对应的物理地址是__pa_symbol(val);

这个可以作为分析问题的辅助手段。

15. 修改系统内存

有时候为了分析问题需要限制系统内存, 可以简单通过修改内核memblock_add函数实现。因为内核伙伴系统的初始化依赖memblock的信息。

memblock_add函数原型如下:

```
int __init_memblock memblock_add(phys_addr_t base, phys_addr_t size)
{
    phys_addr_t end = base + size - 1;
    memblock_dbg("%s: [%pa-%pa] %pS\n", __func__,
                &base, &end, (void *)_RET_IP_);
    return memblock_add_range(&memblock.memory, base, size, MAX_NUMNODES, 0);
}
```

可以通过判断base和size做针对性的调整，类似如下补丁：

```
if (base == 0x0000000100000000)
    return 0;
if (base == 0x0000000009400000)
    size = 0xa6c00000;
```

上面的补丁功能包括：

1. 忽略0x100000000开始的地址空间,因此这片地址空间不会加入系统内存。
2. 修改0x9400000开始的地址空间的size。

16. 内存回收策略调整

有时候为了分析性能和内存异常的问题，需要调整内存交换的策略，主要是对/proc/sys/vm路径下节点的操作。

1. swappiness

swappiness值越大，系统更倾向使用swap分区交换内存，越小，更倾向于使用内存保存数据。

2. dirty_ratio和dirty_background_ratio

控制脏页和总内存的比重,如果超过dirty_ratio，内核主动将脏页写入磁盘，如果超过dirty_background_ratio,后台进程会开始将脏页写入磁盘。

3. min_free_kbytes

指定系统保留的最小空闲内存量，以防止内存完全耗尽，影响系统运行。

有时候为了测试系统稳定性，可以将min_free_kbytes调高，让系统尽早并且频繁的执行回收内存的动作。

4. vfs_cache_pressure

表示内核回收directory和inode cache内存的倾向。

等于100，保持directory和inode cache内存和page cache之间的平衡。

大于100，更倾向于回收directory和inode cache，将更多的内存用于page cache。

小于100，更倾向于保留directory和inode cache，从而减少目录和inode信息的读取次数。

大多数时候这个值不需要测试,除非碰到虚拟文件系统相关性能时候,才需要调整这个参数。

17. 内核各级页表物理地址

以3级页表为例：

第一级页表物理地址 = `__pa_symbol(swapper_pg_dir);`

第二级页表物理地址 = `__pa_symbol(bm_pmd);`

第三级页表物理地址 = `__pa_symbol(bm_pte);`

以上地址用于辅助分析相关问题。

18. slab相关问题分析

1. 内核打开如下配置:

```
CONFIG_SLUB_DEBUG=y
```

```
CONFIG_SLUB_DEBUG_ON=y
```

2. `cat /proc/slabinfo`

统计slub变化情况,假设kmalloc-128异常。

3. 开始trace: `echo 1 > /sys/kernel/slab/kmalloc-128/trace`

执行测试程序。

停止trace: `echo 0 > /sys/kernel/slab/kmalloc-128/trace`

保存trace数据到文件:

```
cat /proc/kmsg > slab-trace-128.log

grep "TRACE kmalloc-128 free" slab-trace-128.log | awk '{print $7}' | sort >
free.txt

grep "TRACE kmalloc-128 alloc" slab-trace-128.log | awk '{print $7}' | sort
> alloc.txt
```

就能统计kmalloc-128相关的分配/释放信息。

19. 内核线性映射的cache属性

内核建立线性映射的函数执行路径大致如下:

```
paging_init -> map_mem -> __map_memblock(pgd, start, end,
pgprot_tagged(PAGE_KERNEL), flags) -> __create_pgd_mapping -> alloc_init_pud -
>init_pmd -> init_pte
```

可以看到使用的prot参数是`pgprot_tagged(PAGE_KERNEL)`,即带MTE(memory tag extension)的cache,所以对kmalloc或者`phys_to_virt`返回的地址进行读写操作都是带cache属性的。

需要说明的是,对于ARM64来说,内核线性映射的建立受

`CONFIG_RODATA_FULL_DEFAULT_ENABLED`配置影响,这个配置关闭的情况下,内核会按照block的方式映射,这个带来的好处是映射的时间缩短,内核页表占据的空间缩小。但是

`CONFIG_RODATA_FULL_DEFAULT_ENABLED`提供了重要的安全保护机制,通过防止对只读数据的修改来增强内核的安全性,默认的`CONFIG_RODATA_FULL_DEFAULT_ENABLED`是打开的。

20. 如何将用户态的地址转换成物理地址

主要是利用`get_user_pages`函数，内核有完整的测试代码，如下：

内核代码：`mm/gup_test.c`

打开内核`CONFIG_GUP_TEST`编译，生成`/sys/kernel/debug/gup_test`节点

用户态测试代码：`tools/testing/selftests/vm/gup_test.c`

进入文件所在目录：`make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- LDFLAGS="-static"`编译生成`gup_test`工具，使用`gup_test`工具测试。

21. io读回来的数据存在一致性问题

`io`命令操作的是`/dev/mem`节点，这个节点默认的映射方式是`write_combined`，可以简单做如下补丁确保`io`正常：

修改`drivers/char/mem.c`当中的`mmap_mem`函数

```
vma->vm_page_prot = pgprot_noncached(vma->vm_page_prot);
```

以上补丁将映射方式修改成`device_memory`，既没有`cache`也没有`write buffer`。

22. ARM32环境预留奇数M的CMA引起内核奔溃

ARM32要求页表按照2M对齐，因此预留CMA空间的时候需要规避这个问题。

23. 用户态访问PCIe BAR空间异常

用户态代码有可能受编译器的优化影响，将多个访问BAR空间的操作优化成一个操作，从而导致地址不对齐。

目前针对这种情况只能根据实际情况调整用户代码，在相应的访问BAR空间的地方加内存屏障`dmb/dsb`防止编译器优化。需要说明的是地址对齐是指地址是访问`size`的整数倍，比如地址是`0xffff0008`，`size`是2/4/8字节。

24. dtb当中预留内存no-map什么意思

内核不会对`no-map`区域的物理内存建立线性映射，意味着要手动为这片内存建立映射，并且不能对这片地址做`cache`操作。

25. 利用/dev/mem访问物理内存

有时候应用程序不想为了访问一个物理内存专门写一个misc驱动，可以利用/dev/mem节点访问，代码片段如下：

CONFIG_DEVMEM打开

CONFIG_STRICT_DEVMEM关闭

```
unsigned long phys_addr = 0x80000000;

unsigned long length = 0x100000;

int fd = open("/dev/mem", O_RDWR);

void *addr = mmap(NULL, length, PROT_READ|PROT_WRITE, MAP_SHARED, fd,
phys_addr);

memset(addr, 0x34, length);

munmap(addr, length);
```

以上代码将物理地址0x80000000 - 0x80100000 1M大小的物理内存写成0x34

26. 内存一致性问题

1. 将映射修改成non-cache验证是否一致性问题。
2. 如果是一致性问题，内核使用dma_sync_single_for_cpu/dma_sync_single_for_device维护cache一致性。

CPU访问内存之前调用dma_sync_single_for_cpu(invalid cache)。

设备访问内存之前调用dma_sync_single_for_device(clean cache)。

需要注意的是dma_sync_single_for_cpu/dma_sync_single_for_device接受的是物理地址。

27. write-combined方式映射的内存存在一致性问题

虽然write-combined映射的内存没有被cache，但是对其写操作有可能被缓存，因此需要使用内存屏障操作dmb/dsb确保数据写到物理内存。

28. dma_alloc_coherent失败

GFP_ATOMIC会导致分配过程走进atomic pool，如果atomic pool配置较小，会引起分配失败，如果遇到这种问题有以下两种尝试：

1. 把GFP_ATOMIC改成GFP_KERNEL。
2. 调整atomic pool大小，内核命令行添加：coherent_pool=4M，设置atomic pool为4MB。

29. DRM如何申请带cache的buffer

用户态代码drm_rockchip_gem_create.flags或者drm_mode_create_dumb.flags需要带上ROCKCHIP_BO_CACHABLE。

30. dma_buf fd的使用方法

主要三步，如下：

```
1. struct dma_buf *buf = dma_buf_get(fd);
2. struct dma_buf_attachment *attach = dma_buf_attach(buf, dev);
3. struct sg_table *table = dma_buf_map_attachment(attach, DMA_BIDIRECTIONAL);
```

31. 生成dma_buf fd的方法

主要有两步,如下：

```
1. struct dma_buf *buf = dma_buf_export(struct dma_buf_export_info *info);
2. dma_buf_fd(buf, O_CLOEXEC);
```

32. 如何配置dma-coherent设备

在dts定义设备节点的时候加入一行dma-coherent；即表示是dma-coherent设备。

dma-coherent意味着设备能够直接访问CPU的cache，不需要软件执行额外的刷cache操作，前提是SoC支持设备dma一致性的访问。

platform设备dma-coherent初始化的路径大致如下：

```
platform_dma_configure -> of_dma_configure -> of_dma_is_coherent ->
arch_setup_dma_ops -> dev->dma_coherent = coherent;
```

当设备调用dma_alloc_coherent申请内存时，cpu的页表属性会从默认的pgprot_writecombine变更为normal，即从non-cached变更为cached。

当设备调用dma_sync_single_for_cpu/dma_sync_single_for_device等接口做cache维护操作时，会先调用dev_is_dma_coherent判断设备是否dma-coherent，如果是则跳过维护cache的操作，因此对于驱动来说是无感的。

33. PCIe设备如何继承PCIe控制器的dma-coherent属性

发生在PCIe设备枚举阶段,以RK芯片为例:

```
rk_add_pcie_port -> dw_pcie_host_init -> pci_host_probe ->
pci_scan_root_bus_bridge -> pci_scan_child_bus -> pci_scan_child_bus_extend ->
pci_scan_slot -> pci_scan_single_device -> pci_scan_device -> pci_device_add ->
device_add -> bus_probe_device -> device_initial_probe -> __device_attach ->
__device_attach_driver -> driver_probe_device ->
__driver_probe_device -> really_probe -> dev->bus->dma_configure ->
pci_dma_configure -> of_dma_configure -> of_dma_is_coherent -> arch_setup_dma_ops
-> dev->dma_coherent = coherent;
```

需要注意的是pci_dma_configure这里使用bridge->parent->of_node作为参数调用of_dma_configure, bridge->parent->of_node就是PCIe控制器节点, 因此PCIe设备继承了PCIe控制器的dma-coherent的属性。

34. 如何分析CMA内存是否被系统借用

可以借助内核trace功能,操作步骤如下:

```
echo 1 > /sys/kernel/debug/tracing/events/kmem/mm_page_alloc_zone_locked/enable

echo 1 > /sys/kernel/debug/tracing/tracing_on
```

以上两步打开mm_page_alloc_zone_locked的trace, mm_page_alloc_zone_locked主要发生在系统从CMA借用page阶段。

重复执行以下命令,也可以写成脚本执行:

```
cat /sys/kernel/debug/tracing/trace | grep migratetype=3
```

如果有借用,会出现类似下面的log:

```
ActivityManager-743      [002] d..3.   315.474016: mm_page_alloc_zone_locked:
page=00000000734f4659 pfn=0x434d6b order=0 migratetype=3 percpu_refill=1
ActivityManager-743      [002] d..3.   315.474017: mm_page_alloc_zone_locked:
page=00000000734f4659 pfn=0x434d6b order=0 migratetype=3 percpu_refill=1
ActivityManager-743      [002] d..3.   315.474018: mm_page_alloc_zone_locked:
page=000000003cebbd4d pfn=0x43378a order=0 migratetype=3 percpu_refill=1
ActivityManager-743      [002] d..3.   315.474019: mm_page_alloc_zone_locked:
page=000000003cebbd4d pfn=0x43378a order=0 migratetype=3 percpu_refill=1
ActivityManager-743      [002] d..3.   315.474021: mm_page_alloc_zone_locked:
page=000000006816a4cc pfn=0x435545 order=0 migratetype=3 percpu_refill=1
ActivityManager-743      [002] d..3.   315.474021: mm_page_alloc_zone_locked:
page=000000006816a4cc pfn=0x435545 order=0 migratetype=3 percpu_refill=1
ActivityManager-743      [002] d..3.   315.474023: mm_page_alloc_zone_locked:
page=00000000a1971528 pfn=0x435547 order=0 migratetype=3 percpu_refill=1
```

CMA的所属的内存物理地址范围是[0x430000000 - 0x440000000], 可以看到进程ActivityManager从CMA的区域借用page, 每次借用一个page, migratetype=3表示CMA的page链表

35. 设置CMA不参与系统内存分配

内核CONFIG_CMA_INACTIVE打开。

36. 透明大页TRANSPARENT_HUGEPAGE

内核打开CONFIG_TRANSPARENT_HUGEPAGE。

在内存富裕环境下，透明大页有助于降低CPU负载，可能带来的问题是：

1. 内存分配延迟
2. 性能可能波动
3. 可能引起内存浪费
4. 可能存在的兼容性问题

用户可通过如下节点开关透明大页功能：

```
cat /sys/kernel/mm/transparent_hugepage/enabled
```

```
always [madvise] never
```

```
echo always > /sys/kernel/mm/transparent_hugepage/enabled
```

37. 无法从缓存中回收内存

cat /proc/meminfo发现Reclaimable还有很多内存，但是内核的分配却失败了，这个大概率是因为申请时表示设置为GFP_ATOMIC，阻止了分配进入内存回收路径，可以尝试修改成GFP_KERNEL，允许内核分配路径进入回收路径。

38. IOVA申请失败

如果npu驱动log提示以下信息：

```
failed to allocate IOVA: -12
```

修改以下代码确认是否是因为size对齐引起的IOVA不足：

```
drivers/iommu/iova.c当中的__alloc_and_insert_iova_range
```

将参数size_aligned强制设置为false，然后再测试

如果以上补丁没有规避问题，可能是IOVA空间确实不满足应用场景，需要从整体减小对IOVA的使用